



Day 8 – Classes & Inheritance in Swift

◆ What I Learned

- Created Classes using `class`
- Learned the difference between Structs and Classes
- Understood that Classes are **Reference Types**
- Used properties and methods inside classes
- Learned about `init()` constructors
- Used `private(set)` for controlled access
- Learned about `deinit`
- Built real-world examples like Bank Account, YouTube Channel, and Game Player
- Learned about Inheritance and Method Overriding
- Understood the `===` identity operator

◆ Basic Class Example

```
class Cat {  
    var name: String = "Billu Don"  
}  
  
let cat1 = Cat()  
  
print(cat1.name)
```

◆ Struct vs Class

Struct	Class
Value Type	Reference Type
Creates copy	Shares same memory
Faster	Slightly slower
Stored in Stack	Stored in Heap
No inheritance	Supports inheritance

◆ Class Example – Reference Type

```
class User {  
    var name: String  
  
    init(name: String) {  
        self.name = name  
    }  
}
```

```
}

var user1 = User(name: "Adarsh")
var user2 = user1

user2.name = "Robin"

print(user1.name)
print(user2.name)
```

Output:

```
Robin
Robin
```

Both variables point to the same object in memory.

◆ Methods Inside Classes

```
class Player {

    var score = 0

    func increaseScore() {
        score += 10
    }
}
```

◆ Initializers (`init`)

Initializers set initial values during object creation.

```
class Car {

    var brand: String

    init(brand: String) {
        self.brand = brand
    }
}
```

◆ Access Control – `private(set)`

`private(set)` allows reading the property but restricts direct modification.

```
private(set) var loading: Bool
```

◆ Deinitializer (`deinit`)

`deinit` runs when a class object is removed from memory.

```
deinit {  
    print("Object removed from memory")  
}
```

Structs do not have `deinit` because they are value types.

◆ Real-World Example – Bank Account

```
class BankAccount {  
  
    var name: String  
    var balance: Int  
  
    init(name: String, balance: Int) {  
        self.name = name  
        self.balance = balance  
    }  
  
    func deposit(amount: Int) {  
        balance += amount  
    }  
  
    func withdraw(amount: Int) {  
  
        if balance >= amount {  
            balance -= amount  
        } else {  
            print("Insufficient Balance")  
        }  
    }  
}
```

◆ Inheritance

Inheritance allows one class to inherit properties and methods from another class.

```
class Vehicle {  
  
    var speed: Int  
  
    init(speed: Int) {  
        self.speed = speed  
    }  
  
    func startEngine() {  
        print("Engine Started")  
    }  
}
```

Child Class:

```
class Car: Vehicle {
```

```
    override func startEngine() {  
        print("Car Engine Started")  
    }  
}
```

◆ Method Overriding

override allows modifying inherited methods.

```
override func startEngine() {  
    print("Car Engine Started")  
}
```

◆ Identity Operator (===)

=== checks whether two class variables point to the same object in memory.

Example:

```
if user1 === user2 {  
    print("Same Object")  
}
```



Important Interview Questions

1 What is a Class in Swift?

A Class is a custom reference type that stores properties and methods together.

2 What is the difference between Struct and Class?

Structs are Value Types.

Classes are Reference Types.

Structs create copies, while Classes share memory references.

3 What is a Reference Type?

A Reference Type shares the same memory location between variables.

4 What does `self` mean in Swift?

`self` refers to the current object instance.

Example:

```
self.name = name
```

5 What is an Initializer?

An Initializer (`init`) sets initial values when creating an object.

6 What is `private(set)`?

`private(set)` allows reading a property publicly but restricts modification outside the class.

7 What is Inheritance?

Inheritance allows one class to inherit properties and methods from another class.

8 What is Method Overriding?

Overriding allows changing inherited method behavior using `override`.

9 What does `===` mean in Swift?

`===` checks whether two variables reference the exact same object in memory.

10 Why are Classes slower than Structs?

Because Classes:

- are stored in Heap memory
- use reference counting
- share memory references

Structs are lighter and faster.

Quick Revision

- Class = Reference Type
- Struct = Value Type
- Classes share memory
- `init` initializes objects
- `deinit` removes objects from memory
- `private(set)` controls modification
- Classes support inheritance
- `override` changes inherited methods
- `===` checks memory reference equality

Day 8 Summary

Today I learned:

- Class creation
- Reference type behavior
- Methods and properties in classes
- Initializers and deinitializers
- Access control using `private(set)`
- Real-world object modeling
- Inheritance
- Method overriding
- Identity operator (`===`)